

Voice-Based Concept Understanding Analyzer

Project Design & Implementation Document

1. Title & Product Overview

Project Name

Voice-Based Concept Understanding Analyser

Project Description

The Voice-Based Concept Understanding Analyser is a single-page Streamlit application that lets a user record or upload a spoken explanation of a technical concept and receive an automated, multi-signal evaluation of how well they explained it. The app combines a **locally run OpenAI Whisper model** for transcription, a **Sentence-BERT semantic-similarity check** against a curated reference explanation for the chosen topic, **Librosa-derived speech metrics** (pause ratio, energy, filler-word rate), and a transparent, **rule-based scoring formula** to produce a 0–100 understanding score. An optional Google Gemini integration — enabled only if the user supplies their own Gemini API key in the sidebar for that session — can auto-detect the topic from the transcript and generate a short block of free-text coaching feedback. Every completed analysis is rendered as an in-app dashboard with a waveform image, is downloadable as a formatted PDF report, and is appended to a session-scoped attempt history the user can revisit from the sidebar.

The platform is built around six functional modules, each implemented as a small, focused Python module:

- **Concept Transcriber (speech_to_text.py):** Loads a local Whisper base model once at import time and transcribes recorded or uploaded audio in either English or Telugu (Beta), also computing a simple filler-word ratio from the resulting text.

- **Semantic Similarity Evaluator (semantic_eval.py):** Holds a fixed library of eight reference topics (Machine Learning, Cloud Computing, Deep Learning, Artificial Intelligence, Neural Networks, Data Science, Natural Language Processing, Computer Vision), each with a short expert-written reference paragraph, and uses Sentence-BERT (all-MiniLM-L6-v2) to compute a single cosine-similarity score between the user's transcript and the selected topic's reference text.
- **Audio Feature Extractor (audio_utils.py):** Uses Librosa to compute duration, RMS energy, zero-crossing rate, and a speech/pause ratio (via silence-interval splitting), and renders a Matplotlib waveform image returned as PNG bytes for display and inclusion in the PDF report.
- **Rule-Based Scoring Engine (scoring_engine.py):** Combines the semantic similarity score, filler-word ratio, pause ratio, and RMS energy into a weighted 0–100 understanding score with a three-tier classification (Strong / Moderate / Poor Understanding), plus a separate fluency sub-score derived from filler and pause ratios.
- **Optional Gemini Add-ons (semantic_eval.py):** Two independent, best-effort Gemini calls — auto_detect_topic(), which classifies the transcript into one of the eight fixed topics, and get_gemini_feedback(), which generates a short block of free-text coaching feedback — both gated behind a Gemini API key the user types into the sidebar for that session only.
- **PDF Report Generator (report_generator.py):** Uses ReportLab to assemble a formatted PDF containing the reference concept text, the full transcript, the waveform image, a metrics summary table, and the Gemini feedback (or a placeholder message if no key was supplied).

Together, these modules give a user immediate, multi-angle feedback on a spoken explanation — an objective similarity-to-reference score, concrete speech-delivery metrics, and an overall understanding rating — with optional LLM-generated coaching layered on top, all running as a single local Streamlit app.

Technology Stack (as implemented)

Layer	Technology	Purpose
UI + Application Logic	Streamlit	Single-script app (app.py) rendering the sidebar, recorder/uploader, and results dashboard, and orchestrating the full pipeline on each button click
Audio Recording Widget	audio-recorder-streamlit (audio_recorder())	Renders an in-browser microphone recorder button and returns raw audio bytes
Speech-to-Text	OpenAI Whisper, local inference (whisper.load_model("base"))	Transcribes recorded/uploaded audio on-device in English or Telugu (Beta); model is loaded once at module import
Semantic Similarity	Sentence-BERT (sentence-transformers, all-MiniLM-L6-v2)	Computes a single cosine-similarity score between the transcript and a fixed reference paragraph for the selected topic
Audio Feature Extraction & Waveform	Librosa + Matplotlib	Computes duration, RMS energy, zero-crossing rate, and pause ratio; renders a waveform plot as PNG bytes
Scoring	Custom rule-based Python logic (scoring_engine.py)	Deterministic weighted formula (no LLM involved) producing the final 0-100 score, understanding level, and fluency sub-score
Optional Reasoning Add-ons	Google Gemini API (gemini-1.5-flash)	Best-effort topic auto-detection and free-text coaching feedback; only runs if the user supplies an API key in the sidebar

PDF Report Generation	ReportLab (SimpleDocTemplate, Platypus flowables)	Generates a downloadable PDF with the reference text, transcript, waveform image, metrics table, and feedback
Attempt History	Streamlit st.session_state (in-memory list)	Stores attempts for the current browser session only; not persisted to disk or a database, and is cleared on refresh/restart or via the sidebar "Clear History" button
NLP Utility	nltk (punkt tokenizer download)	Downloaded at import time in speech_to_text.py; available for future sentence-level processing
Deployment	Streamlit local (streamlit run app.py)	Runs the app on http://localhost:8501

Known Implementation Notes & Constraints

These are drawn directly from the current codebase and are worth documenting explicitly rather than glossing over:

- **Topic list is fixed, not open-ended:** CONCEPT_REFERENCES in semantic_eval.py hardcodes exactly eight topics. A user explaining a concept outside this list must either pick the closest match manually or rely on Gemini auto-detection, which still snaps to one of the same eight topics.
- **Gemini key is entered per session, not loaded from .env:** Although a .env.example file exists in the repository (GEMINI_API_KEY=your_gemini_api_key_here), the current app.py does not use python-dotenv — the key is collected via a sidebar st.text_input(type="password") and used only in memory for that session.
- **Attempt history does not survive a restart:** st.session_state.history is an in-memory Python list scoped to the active Streamlit session; there is no SQLite or file-based persistence, so history disappears on a full app restart or a new browser session.
- **ffmpeg discovery includes a hardcoded personal path:** get_ffmpeg_path() in speech_to_text.py falls back to a specific WinGet installation path on one developer's machine (C:\Users\sujth\...) before trying generic common paths — this should be treated as a portability issue to clean up before wider distribution.
- **No explicit audio size/duration validation:** unlike a hardened production pipeline, the current code does not reject overly long or oversized recordings before sending them to Whisper/Librosa; very long files will simply take longer to process.
- **Scoring is deterministic and LLM-free:** the 0-100 score itself is never produced by Gemini — it comes entirely from evaluate_understanding()'s fixed point thresholds, which makes the score reproducible and free of LLM cost, at the expense of the nuanced, per-gap reasoning an LLM-driven rubric could offer.

2. Scenarios

Scenario 1: A student opens the sidebar, leaves "Auto Detect Topic" unchecked, selects "Machine Learning" from the topic dropdown, and sees the built-in reference paragraph displayed in the right-hand column. They record a 45-second explanation using the microphone button under the "Live Record" tab. On clicking "Analyze Concept Understanding," the local Whisper model transcribes the recording, `filler_word_ratio()` finds a moderate number of filler words, `extract_audio_features()` reports a pause ratio of 0.18 and healthy RMS energy, and `semantic_similarity()` returns 0.74 against the Machine Learning reference paragraph. `evaluate_understanding()` combines these into a score of 85/100 and a "Strong Understanding" $\square$ classification, with a fluency score of 78/100. Because no Gemini key was entered, the feedback panel simply shows "Add Gemini API key for AI feedback." The attempt is appended to the sidebar's Attempt History list and a PDF report is available for download.

Scenario 2: A user enables "Auto Detect Topic (Gemini)" and pastes a Gemini API key into the sidebar, then uploads a pre-recorded .mp3 file explaining "Neural Networks" via the "Upload File" tab without selecting a topic manually. After transcription, `auto_detect_topic()` sends the transcript to `gemini-1.5-flash`, which classifies it as "Neural Networks" from the fixed eight-topic list; `get_reference()` then supplies the matching reference paragraph for the similarity check. The resulting similarity is only 0.41 (moderate topical overlap but imprecise phrasing), pushing the rule-based score to 52/100 — a "Moderate Understanding" $\square$ result. Because a Gemini key was supplied, `get_gemini_feedback()` also returns three to four sentences of free-text coaching (e.g., encouraging more precise terminology around activation functions), which is shown on screen and included in the downloaded PDF report.

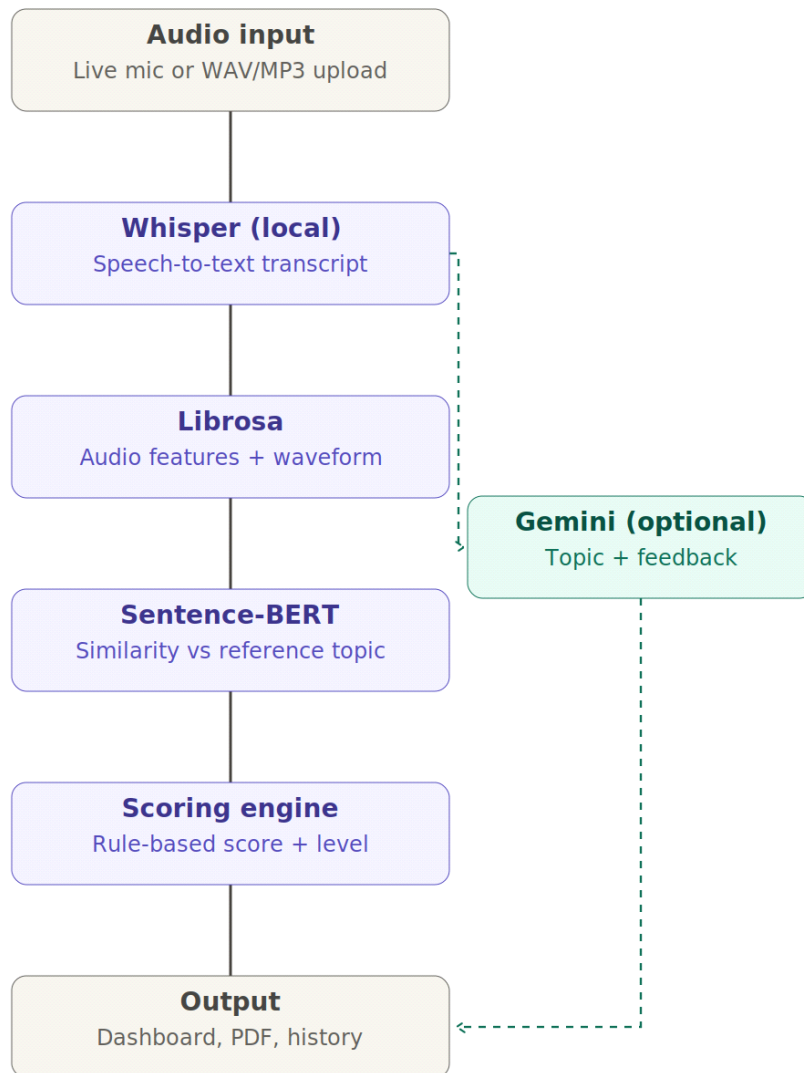
Scenario 3: A user selects the Telugu (Beta) language option and records an explanation of "Cloud Computing" in Telugu. `speech_to_text()` passes `language="te"` into Whisper's `transcribe()` call; because the base English-tuned model has weaker multilingual accuracy, the resulting transcript is noisier than an equivalent English recording, which the UI flags with a standing sidebar warning ("Telugu support is in Beta"). The semantic similarity comparison still runs against the English-language Machine Learning-style reference paragraph library, so the similarity score is likely to be lower purely due to the cross-language comparison — an explicit, documented limitation of the current implementation rather than a bug, since there is no Telugu-language reference text or translation step in the pipeline yet.

3. Technical Architecture

3.1 Architecture Overview

Description: The application is a single-tier Streamlit-based system where `app.py` coordinates all modules sequentially when the **Analyze Concept Understanding** button is clicked, storing temporary data in `st.session_state`.

- **UI Layer (`app.py`):** Manages the Streamlit interface, settings, audio input, results, history, waveform, and PDF download.
- **Recording Layer (`audio_recorder_streamlit`):** Captures microphone audio as WAV using `audio_recorder_streamlit`.
- **Transcription Layer (`speech_to_text.py`):** Uses a local Whisper model to transcribe speech and calculate filler-word ratio.
- **Audio Feature Layer (`audio_utils.py`):** Extracts speech features and generates waveform images with Librosa and Matplotlib.
- **Semantic Evaluation Layer (`semantic_eval.py`):** Uses Sentence-BERT to measure semantic similarity and optionally Gemini for topic detection and feedback.
- **Scoring Layer (`scoring_engine.py`):** Calculates understanding and fluency scores and assigns performance levels.
- **Reporting Layer (`report_generator.py`):** Creates a PDF report containing the transcript, metrics, waveform, scores, and Gemini feedback.



Purple = core pipeline · Teal = optional, needs a Gemini key · Gray = start / end

3.2 Processing Flow (as implemented)

```

audio_recorder() / st.file_uploader() -> saved to a temp .wav/.mp3
-> speech_to_text(audio_path, lang_code) [local Whisper] -> transcript
-> filler_word_ratio(transcript) -> filler ratio
-> extract_audio_features(audio_path) [Librosa] -> duration, RMS, pause
ratio
-> save_waveform(audio_path) [Matplotlib] -> waveform PNG bytes
-> [if auto_detect] auto_detect_topic(transcript, api_key) -> topic
-> get_reference(topic) -> semantic_similarity() [Sentence-BERT] -> similarity score
-> evaluate_understanding() + calculate_fluency_score() -> score, level, fluency
-> [if key present] get_gemini_feedback() -> feedback text
-> st.session_state.history / .result updated
-> Streamlit renders dashboard -> generate_pdf_report() -> st.download_button
  
```

3.3 Component Responsibilities

Component	File	Key Functions
app.py	App root	Page config, sidebar, tabs, Analyze button handler, results rendering
speech_to_text.py	Module	get_ffmpeg_path(), speech_to_text(), get_word_count(), get_speaking_rate(), filler_word_ratio()
semantic_eval.py	Module	CONCEPT_REFERENCES, semantic_similarity(), auto_detect_topic(), get_gemini_feedback(), get_reference()
audio_utils.py	Module	extract_audio_features(), save_waveform()
scoring_engine.py	Module	evaluate_understanding(), get_score_emoji(), calculate_fluency_score()
report_generator.py	Module	generate_pdf_report()

3.4 In-Memory Data Shape (Attempt Result)

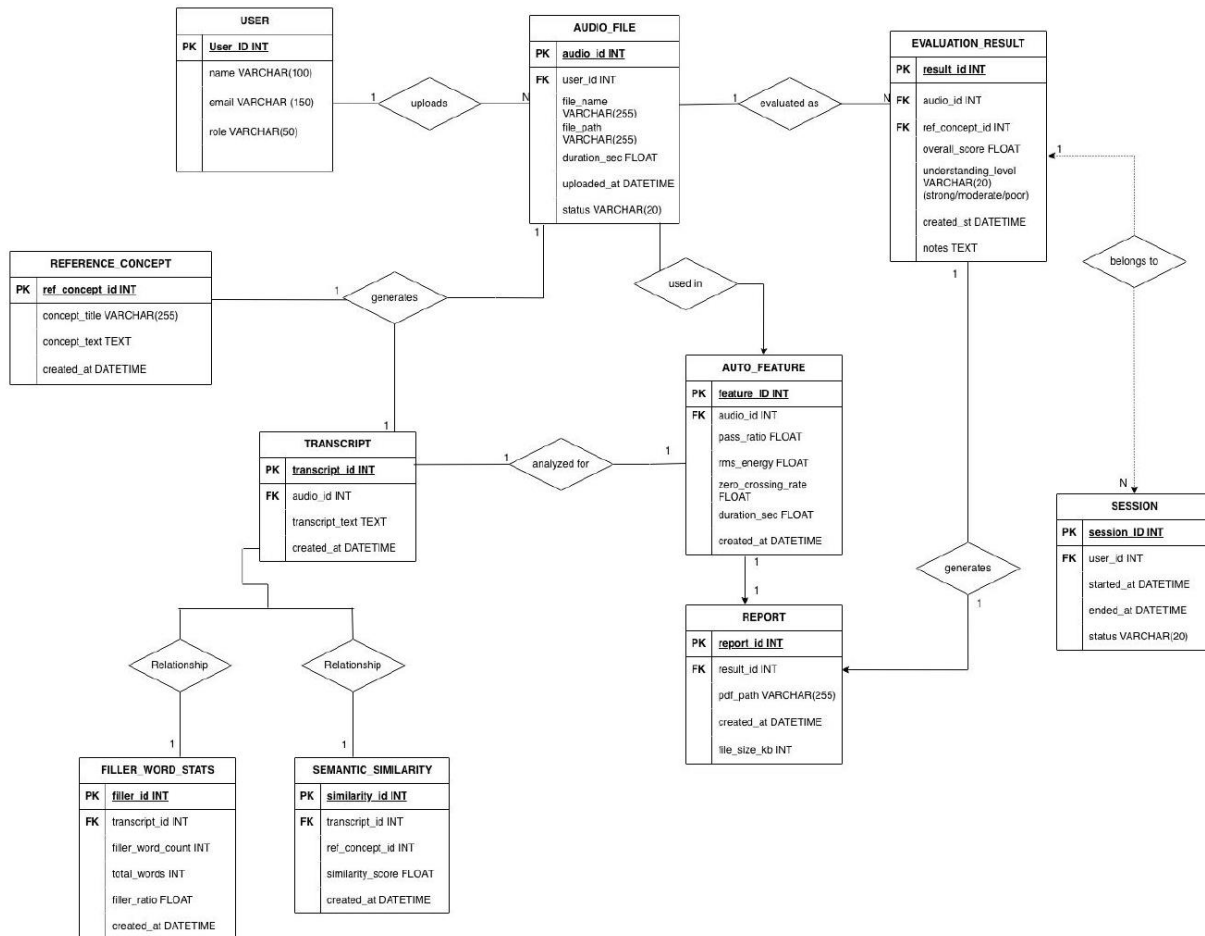
There is no database schema in the current implementation — each attempt is a plain Python dict held in `st.session_state.result` and appended (nested under a summary dict) to `st.session_state.history`. This is the shape actually produced by `app.py`:

```
{
  "transcript": "string", "topic": "Machine Learning", "similarity": 0.74, "filler": 0.045,
  "audio_features": {"duration": 46.2, "rms_energy": 0.0312, "pause_ratio": 0.18},
  "score": 85, "level": "Strong Understanding", "color": "#2ecc71",
  "fluency": {"fluency_score": 78.0, "fluency_level": "Fluent"},
  "feedback": "Add Gemini API key for AI feedback.", "waveform_bytes": "<PNG bytes>"
}
```

Persistence caveat: because this structure lives only in `st.session_state`, it is lost whenever the Streamlit process restarts or the browser session ends. Section 6 (Future Scope) proposes an SQLite-backed history table as a natural upgrade path that would not require changing this in-memory shape, only adding a save/load layer around it.

3.5 Proposed Database Schema (Future Persistence Layer)

The ER diagram below is the proposed relational schema for the SQLite-backed persistence upgrade referenced in Section 6 (Future Scope). It is not yet implemented in the current codebase — today's `st.session_state` dict (Section 3.4) would map onto this schema as follows once a database layer is added.



Proposed ER diagram for persisting users, audio files, transcripts, evaluation results, and reports.

- **USER (1 -> N AUDIO_FILE)**: a user uploads or records many audio files; role distinguishes student/interview-candidate style usage if multi-user support is added.
- **AUDIO_FILE (1 -> N TRANSCRIPT, 1 -> N AUTO_FEATURE, 1 -> N EVALUATION_RESULT)**: each recording maps to today's audio_path, and status tracks pipeline progress (uploaded -> transcribed -> scored).
- **TRANSCRIPT (1 -> 1 FILLER_WORD_STATS, 1 -> 1 SEMANTIC_SIMILARITY)**: splits the current combined result dict into normalized tables — filler_ratio and the similarity score each get their own row instead of sibling keys in one dict.
- **REFERENCE_CONCEPT**: normalizes the current hardcoded CONCEPT_REFERENCES dictionary into rows, which directly supports the Future Scope goal of supporting arbitrary/generated topics instead of a fixed eight.
- **EVALUATION_RESULT**: corresponds to today's score, level, and fluency fields, now linked back to both the audio_id and the ref_concept_id it was scored against.
- **REPORT (N -> 1 EVALUATION_RESULT)**: persists the pdf_path of each generate_pdf_report() export, letting a user re-download a past report without regenerating it.
- **SESSION (1 -> N per USER)**: an optional grouping entity for tracking multiple attempts within one sitting, useful for the score-trend charts proposed in Future Scope.

4. Pre-requisites

- **Streamlit Framework Knowledge:** Streamlit Documentation (docs.streamlit.io)
- **audio-recorder-streamlit component:** PyPI project page (pypi.org/project/audio-recorder-streamlit)
- **OpenAI Whisper (local) Familiarity:** Whisper GitHub Repository (github.com/openai/whisper)
- **Sentence-Transformers (Sentence-BERT):** sentence-transformers Documentation (sbert.net)
- **Google Gemini API Familiarity (optional feature):** Google AI Studio / Gemini API Documentation (ai.google.dev/docs)
- **Librosa for Audio Processing:** Librosa Documentation (librosa.org/doc)
- **ffmpeg (required by Whisper for audio decoding):** FFmpeg Documentation (ffmpeg.org)
- **ReportLab for PDF Generation:** ReportLab User Guide (docs.reportlab.com)
- **NLTK (punkt tokenizer):** NLTK Documentation (nltk.org)
- **Python Programming Proficiency:** Python Documentation (docs.python.org)
- **Version Control with Git:** Git Documentation (git-scm.com/doc)
- **PyTorch (Whisper + Sentence-BERT dependency):** PyTorch Documentation (pytorch.org/docs)

5. Project Workflow & Milestones

Milestone 1: Model Selection and Environment Setup

This milestone covers selecting the local Whisper model and Sentence-BERT model actually used in the codebase, and setting up the environment needed to run app.py.

Activity 1.1: (Optional) Obtain a Gemini API Key

Because the current implementation collects the Gemini key directly through the Streamlit sidebar rather than from a .env file, this step is optional and per-session — a user can run the entire app, including transcription, scoring, waveform, and PDF export, without ever entering a Gemini key. The key is only needed to unlock the two optional Gemini features (topic auto-detection and free-text feedback).

- **Step 1:** Visit aistudio.google.com and sign in with a Google account.
- **Step 2:** Click "Get API key," then "Create API key," choosing a new or existing Google Cloud project.
- **Step 3:** Copy the generated key.
- **Step 4:** Paste it into the "Gemini API Key" password field in the app's sidebar at runtime — it is held only in that session's Streamlit state and is never written to disk by the current code.

A .env.example file is included in the repository for future use (see Section 3, Known Implementation Notes), but the current app.py does not load it — GEMINI_API_KEY is not read from the environment anywhere in the codebase today.

```
GEMINI_API_KEY=your_gemini_api_key_here
```

Activity 1.2: Model Selection (as implemented)

- **Whisper model:** `whisper.load_model("base")` is called once at the top of `speech_to_text.py`, outside any function, so it loads a single time when the module is first imported and is reused for every transcription for the life of the process.
- **Sentence-BERT model:** `SentenceTransformer('all-MiniLM-L6-v2')` is likewise instantiated once at the top of `semantic_eval.py`.
- **Gemini model:** `genai.GenerativeModel('gemini-1.5-flash')` is created fresh inside both `auto_detect_topic()` and `get_gemini_feedback()` each time they are called, using whatever API key was passed in from the sidebar for that call.

Activity 1.3: Set Up the Development Environment

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

requirements.txt: streamlit, openai-whisper, sentence-transformers, librosa, numpy, matplotlib, reportlab, torch, nltk, google-generativeai, audio-recorder-streamlit.

Install ffmpeg (required by Whisper for audio decoding): download the official build on Windows and add it to PATH, brew install ffmpeg on macOS, or apt-get install ffmpeg on Linux. `speech_to_text.py`'s `get_ffmpeg_path()` will also try to auto-discover it (see Milestone 2, Activity 2.1).

Project Structure: `app.py`, `speech_to_text.py`, `semantic_eval.py`, `audio_utils.py`, `scoring_engine.py`, `report_generator.py`, `requirements.txt`, and `.env.example` under "5. Project Development Phase"; `test_functional.py` and `test_performance.py` under "6. Project Testing".

Milestone 2: Core Module Implementation (as built)

Activity 2.1: Concept Transcription and Filler Detection (*speech_to_text.py*)

This module locates ffmpeg, loads the local Whisper base model once, and exposes transcription plus a few simple text-metric helpers.

```
model = whisper.load_model("base") # loaded once at import time

def speech_to_text(audio_path, language="en"):
    result = model.transcribe(audio_path, language=language, task="transcribe")
    return {"success": True, "text": result["text"].strip()}

def filler_word_ratio(text):
    fillers = ["um", "uh", "like", "basically", "actually", "literally", "so"]
    words = text.lower().split()
    return round(sum(1 for w in words if w in fillers) / len(words), 4)
```

Activity 2.2: Semantic Similarity and Optional Gemini Add-ons (*semantic_eval.py*)

```
model = SentenceTransformer('all-MiniLM-L6-v2')

CONCEPT_REFERENCES = {
    "Machine Learning": "Machine learning is a subset of AI that enables systems to learn...",
    # ...7 more fixed topics (Cloud Computing, Deep Learning, AI, Neural Networks, etc.)
}

def semantic_similarity(text1, text2):
    embeddings = model.encode([text1, text2], convert_to_tensor=True)
    return round(float(util.cos_sim(embeddings[0], embeddings[1])), 4)

def auto_detect_topic(text, api_key):
    try:
        gemini = genai.GenerativeModel('gemini-1.5-flash')
        detected = gemini.generate_content(f'Best topic for: "{text}"?').text.strip()
        return detected if detected in CONCEPT_REFERENCES else "Machine Learning"
    except:
        return "Machine Learning" # silent fallback - see Section 8
```

Note on error handling: `auto_detect_topic()` uses a bare `except:` clause that silently falls back to "Machine Learning" on any failure (invalid key, network error, unexpected response format). This is convenient for a demo but means a bad Gemini key produces a silently wrong topic rather than a visible error — flagged further in Section 8 (Security Considerations).

Activity 2.3: Audio Feature Extraction and Waveform Rendering (*audio_utils.py*)

```
def extract_audio_features(audio_path):
    y, sr = librosa.load(audio_path, sr=None)
    duration = librosa.get_duration(y=y, sr=sr)
    rms_energy = float(np.mean(librosa.feature.rms(y=y)))
    speech_duration = sum((e - s) / sr for s, e in librosa.effects.split(y, top_db=30))
    pause_ratio = round((duration - speech_duration) / duration, 4)
    return {"duration": duration, "rms_energy": rms_energy, "pause_ratio": pause_ratio}

def save_waveform(audio_path):
    y, sr = librosa.load(audio_path, sr=None)
    plt.plot(np.linspace(0, len(y) / sr, len(y)), y, color='#00b4d8', linewidth=0.5)
    # ...dark theme styling + savefig to PNG bytes, returned via BytesIO
```

Note: `save_waveform()` re-loads the audio file from disk with Librosa independently of `extract_audio_features()`, rather than reusing an already-decoded signal — a straightforward efficiency improvement would be to decode once and pass the signal into both functions (see Section 6, Future Scope).

Activity 2.4: Rule-Based Scoring Engine (*scoring_engine.py*)

```
def evaluate_understanding(similarity, filler_ratio, audio):
    score = 0
    score += 50 if similarity > 0.7 else 30 if similarity > 0.4 else 10 # similarity: 50 pts
    score += 20 if filler_ratio < 0.05 else 10 # filler words: 20 pts
    score += 15 if audio["pause_ratio"] < 0.25 else 5 # pause ratio: 15 pts
    score += 15 if audio["rms_energy"] > 0.01 else 5 # RMS energy: 15 pts
    level = "Strong" if score >= 80 else "Moderate" if score >= 50 else "Poor"
    return score, f"{level} Understanding"

def calculate_fluency_score(filler_ratio, pause_ratio):
    fluency = max(0, min(100, round(100 - filler_ratio * 200 - pause_ratio * 100, 2)))
    return {"fluency_score": fluency}
```

Scoring transparency: because every threshold here is a fixed Python constant (0.7, 0.4, 0.05, 0.25, 0.01, 80, 50), the entire scoring model is fully auditable and reproducible — the same transcript and audio metrics will always yield the same score. This is a deliberate trade-off against the more adaptive, per-gap reasoning an LLM-based rubric could provide.

Milestone 3: Application Orchestration (*app.py*)

Milestone 3 covers *app.py* itself — the single Streamlit script that wires together every module from Milestone 2, manages session state, and renders the sidebar and results UI. The code below is reproduced from the repository as committed.

Activity 3.1: Page Setup, Styling, and Session State

```
st.set_page_config(page_title="Voice-Based Concept Understanding Analyser", layout="wide")
st.title("Voice-Based Concept Understanding Analyser")

if "history" not in st.session_state: st.session_state.history = []
if "result" not in st.session_state: st.session_state.result = None
```

Activity 3.2: Sidebar — Settings, Topic Selection, and History

```
gemini_api_key = st.sidebar.text_input("Gemini API Key", type="password")
auto_detect = st.sidebar.checkbox("Auto Detect Topic (Gemini)")
topic = st.sidebar.selectbox("Select Concept Topic", list(CONCEPT_REFERENCES.keys()))
language = st.sidebar.radio("Language", ["English", "Telugu (Beta)"])

for i, h in enumerate(st.session_state.history[-5:]):
    if st.sidebar.button(f"Attempt {i+1}: {h['topic']} -> {h['score']}/100"):
        st.session_state.result = h['result']
```

Note: the last five attempts are shown as clickable sidebar buttons (`[-5:]` slice); clicking one restores that attempt's result dict into `st.session_state.result` without re-running any analysis, since the full result was cached at the time of the original attempt.

Activity 3.3: Main Area — Recorder, Uploader, and Reference Panel

```
tab1, tab2 = st.tabs(["Live Record", "Upload File"])
with tab1:
    audio_bytes = audio_recorder()
with tab2:
    audio_file = st.file_uploader("Upload audio (WAV/MP3)", type=["wav", "mp3"])

if not auto_detect:
    st.info(get_reference(topic)) # Concept Reference panel
```

Activity 3.4: Analyze Button — Full Pipeline Orchestration

```
transcript = speech_to_text(audio_path, lang_code)["text"]
filler = filler_word_ratio(transcript)
audio_features = extract_audio_features(audio_path)
waveform_bytes = save_waveform(audio_path)

current_topic = auto_detect_topic(transcript, gemini_api_key) if auto_detect else topic
similarity = semantic_similarity(transcript, get_reference(current_topic))
score, level = evaluate_understanding(similarity, filler, audio_features)
fluency = calculate_fluency_score(filler, audio_features["pause_ratio"])
feedback = get_gemini_feedback(transcript, current_topic, score, gemini_api_key)

result = {..., "topic": topic} # <- saved to history
st.session_state.history.append({"topic": topic, "score": score, "result": result})
result["topic"] = current_topic # <- but shown as live result
st.session_state.result = result # (topic mismatch bug)
```

Note on the topic mismatch: when `auto_detect` is enabled, the code stores the originally selected sidebar topic (`topic`) into `st.session_state.history`, but stores the Gemini-detected topic (`current_topic`) into `st.session_state.result`. This is a real inconsistency worth fixing (see Section 6, Future Scope) — the sidebar history label and the live result panel can show two different topic names for the same attempt when auto-detection changes the topic.

Note on temp-file cleanup: the finally block only deletes the temporary file if it came from the uploader path (`audio_path != recorded_path`); a live-recorded temp file (`recorded_path`) is deliberately left on disk so the sidebar's "Attempt History" buttons can still reference it later in the session, but this means recorded .wav temp files accumulate in the OS temp directory for the life of the process (see Section 8, Security Considerations).

Activity 3.5: Results Rendering

```
r = st.session_state.result
st.write(r["transcript"])
st.markdown(f"<h1 style='color:{r['color']}'>{r['score']}/100</h1>", unsafe_allow_html=True)

st.metric("Semantic Similarity", r["similarity"])
st.metric("Filler Words", r["filler"])
st.metric("Fluency Score", f"{r['fluency']['fluency_score']}/100")
st.image(r["waveform_bytes"], use_column_width=True)

pdf_bytes = generate_pdf_report(topic=r['topic'], reference_text=r['ref_text'], ...)
st.download_button("Download Full PDF Report", data=pdf_bytes,
    file_name=f"VBCUA_Report_{r['topic'].replace(' ', '_')}.pdf", mime="application/pdf")
```

Milestone 4: PDF Report Generation (report_generator.py)

Milestone 4 covers the ReportLab-based PDF export, which assembles the reference text, transcript, waveform image, a metrics summary table, and the Gemini feedback (or its placeholder) into a single downloadable document.

```
def generate_pdf_report(topic, reference_text, transcribed_text, final_score,
                        understanding_level, gemini_feedback, waveform_bytes=None):
    doc = SimpleDocTemplate(buffer, pagesize=letter)
    styles = getSampleStyleSheet()
    story = [
        Paragraph("Voice-Based Concept Understanding Analyser – Evaluation Report",
styles['Heading1']),
        Paragraph(f"<b>Topic:</b> {topic}", styles['Normal']),
        Paragraph(reference_text, styles['Normal']),
        Paragraph(transcribed_text, styles['Normal']),
        Image(io.BytesIO(waveform_bytes), width=6 * inch, height=2 * inch),
        Table([["Metric", "Value"], ["Final Score", f"{final_score}/100"],
["Understanding Level", understanding_level]]),
        Paragraph(gemini_feedback, styles['Normal']),
    ]
    doc.build(story)
    return buffer.getvalue()
```

Milestone 5: Deployment (Streamlit Local)

The application is run locally via Streamlit's built-in development server. There is no Flask/Gunicorn layer, no Ngrok tunnel, and no containerization in the current repository — deployment is simply running the script.

Activity 5.1: Prepare the Environment

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

Ensure ffmpeg is discoverable: either install it and add it to the system PATH so `shutil.which("ffmpeg")` in `get_ffmpeg_path()` finds it directly, or be aware that the code will otherwise fall back to a hardcoded developer-specific path that will not exist on other machines (see Section 3 and Section 8).

Activity 5.2: Run the App

```
(myenv) D:\projects>cd "5. Project Development Phase"
(myenv) D:\projects\5. Project Development Phase>streamlit run app.py
```

Streamlit starts its local development server and opens the app in the default browser:

```
Local URL: http://localhost:8501
Network URL: http://192.168.x.x:8501
```

The first run will download the local Whisper base model (~140MB) and the Sentence-BERT all-MiniLM-L6-v2 model (~80MB) to the default Hugging Face / Whisper cache directories; subsequent runs load from cache.

Activity 5.3: Local Testing and Verification

Test Case	Expected Result (as implemented)
-----------	----------------------------------

Record a clear explanation, topic selected manually, no Gemini key	Full results render; feedback shows "Add Gemini API key for AI feedback."
Same, with a valid Gemini key and Auto Detect Topic checked	Topic is inferred from the transcript and matched against the fixed CONCEPT_REFERENCES list; free-text feedback appears
Upload a .mp3 file instead of recording	audio_file.type branch selects the .mp3 suffix and transcription proceeds identically
Select Telugu (Beta) and record in Telugu	speech_to_text() is called with language="te"; sidebar warning is shown; similarity is still computed against the English reference paragraph
Click Analyze with neither a recording nor an upload present	st.error("Please record or upload an audio file.") and st.stop() halt the handler
Enter an invalid Gemini API key with Auto Detect Topic checked	auto_detect_topic()'s bare except swallows the error and silently falls back to "Machine Learning" rather than surfacing a message
Click a past attempt in the sidebar Attempt History	st.session_state.result is restored from history without re-running transcription or scoring
Click "Clear History"	st.session_state.history is emptied; the currently displayed result (if any) is unaffected
Restart the Streamlit process	Attempt History is empty again, since it was only ever held in st.session_state

Important Notes

- There is no public URL, Ngrok tunnel, or reverse proxy involved — the app is reachable only at <http://localhost:8501> (and the printed Network URL, for other devices on the same LAN).
- Because Whisper's base model and Sentence-BERT's all-MiniLM-L6-v2 are both loaded once at module import time (not behind a Streamlit cache decorator), Streamlit's script-rerun model means these imports only execute on the very first script run per process, not per interaction — subsequent reruns reuse the already-imported module-level model objects.
- The `get_ffmpeg_path()` hardcoded fallback path should be removed or replaced with an environment-variable-driven path before this app is shared with or run by anyone other than the original developer (see Section 8).

Milestone 6: Conclusion & Future Scope

The Voice-Based Concept Understanding Analyser, as implemented, delivers a working end-to-end pipeline: local Whisper transcription, Sentence-BERT semantic similarity against a curated reference library, Librosa-derived speech-delivery metrics, a transparent rule-based scoring formula, an optional Gemini-powered coaching layer, a downloadable ReportLab PDF, and a session-scoped attempt history — all inside a single Streamlit script with no external server dependency. It successfully demonstrates the core product idea ("explain a concept out loud and get scored feedback") without requiring any paid API key for its core functionality, since Gemini is strictly additive.

At the same time, reviewing the implementation surfaces a clear, concrete list of near-term improvements, several of which are minor code fixes rather than new features:

- **Fix the topic-mismatch bug:** store the same topic value (`current_topic`, not the original sidebar topic) in both `st.session_state.history` and `st.session_state.result` so the sidebar history label always matches the analyzed topic, especially when Auto Detect Topic changes it.

- **Persist attempt history to SQLite:** replace the in-memory `st.session_state.history` list with the normalized schema proposed in Section 3.5 (`USER`, `AUDIO_FILE`, `TRANSCRIPT`, `EVALUATION_RESULT`, `REPORT`, etc.) so history survives an app restart.
- **Remove the hardcoded ffmpeg path:** replace the developer-specific `WinGet` path in `get_ffmpeg_path()` with an environment variable (e.g., `FFMPEG_PATH`) or drop the fallback entirely in favor of a clear installation error message.
- **Support arbitrary topics:** extend beyond the fixed eight-entry `CONCEPT_REFERENCES` dictionary, for example by having Gemini generate an on-the-fly reference paragraph for a user-typed topic when no matching entry exists.
- **Surface Gemini failures instead of swallowing them:** replace the bare `except:` in `auto_detect_topic()` with a specific exception type and a visible `st.warning()` so an invalid API key doesn't silently and misleadingly default to "Machine Learning."
- **Load .env for the Gemini key as a convenience option:** honor the already-present `.env.example` by optionally pre-filling the sidebar key field from `python-dotenv` if `GEMINI_API_KEY` is set, while still allowing per-session override.
- **Avoid double-decoding audio:** have `save_waveform()` accept an already-loaded signal from `extract_audio_features()` instead of reloading the file independently with `Librosa`.
- **Add basic input limits:** reject or warn on unusually long recordings before sending them to `Whisper`, to bound processing time on constrained machines.
- **True multilingual reference comparison:** for the Telugu (Beta) path, add Telugu-language reference paragraphs (or a translation step) so the semantic similarity comparison is not systematically penalized by a cross-language mismatch.

Longer-term, the same modular structure (separate transcription, scoring, audio-feature, and reporting modules) would support layering in more advanced analysis — such as the earlier-explored idea of LLM-driven, per-gap knowledge assessment — without discarding any of the existing rule-based scoring, simply by adding it as an additional, clearly-labeled signal alongside the current score.

6. Detailed Website Walkthrough Descriptions

Main Page Layout

Description: The app opens directly into its single page (no separate landing/results routes). A dark-themed header (#0d1117 background, white text, styled via an inline `st.markdown` block) shows the title "Voice-Based Concept Understanding Analyser" and the subtitle "Automated evaluation of spoken conceptual explanations using AI," followed by a horizontal rule. Below that, the page splits into a wide left sidebar and a two-column main body.

Sidebar — Settings and History

Description: The sidebar (`st.sidebar`) holds every configuration control: a password-masked Gemini API Key field, an "Auto Detect Topic (Gemini)" checkbox, a topic dropdown (shown only when auto-detect is off, populated from the eight `CONCEPT_REFERENCES` keys), and a language radio (English / Telugu (Beta)) with a standing warning banner when Telugu is selected. Below a divider, an "Attempt History" section lists the five most recent attempts as clickable buttons labeled "Attempt N: {topic} -> {score}/100"; clicking one instantly restores that attempt's full result panel. Two more buttons — "Reset / New Analysis" and "Clear History" — round out the sidebar.

Main Area — Recording, Upload, and Reference Panel

Description: The main body is split into a wider left column (1.5x) and a narrower right column. The left column contains two tabs: "Live Record," which renders the `audio_recorder()` microphone button and, once a recording is captured, an inline `st.audio()` playback widget; and "Upload File," a standard `st.file_uploader` restricted to WAV/MP3. The right column, visible only when Auto Detect Topic is off, shows a "Concept Reference" heading followed by an `st.info()` box containing the full reference paragraph for whichever topic is currently selected in the sidebar dropdown — giving the user a preview of what a strong answer should cover before they even record.

Analyze Action and Processing State

Description: A single primary-styled "Analyze Concept Understanding" button sits below the two-column area. Clicking it wraps the entire pipeline in one `st.spinner("Transcribing and evaluating...")`, so the user sees a single continuous loading indicator rather than a staged, multi-step progress message — transcription, feature extraction, similarity scoring, and (optionally) the two Gemini calls all happen inside that one spinner context before anything else re-renders.

Results Section

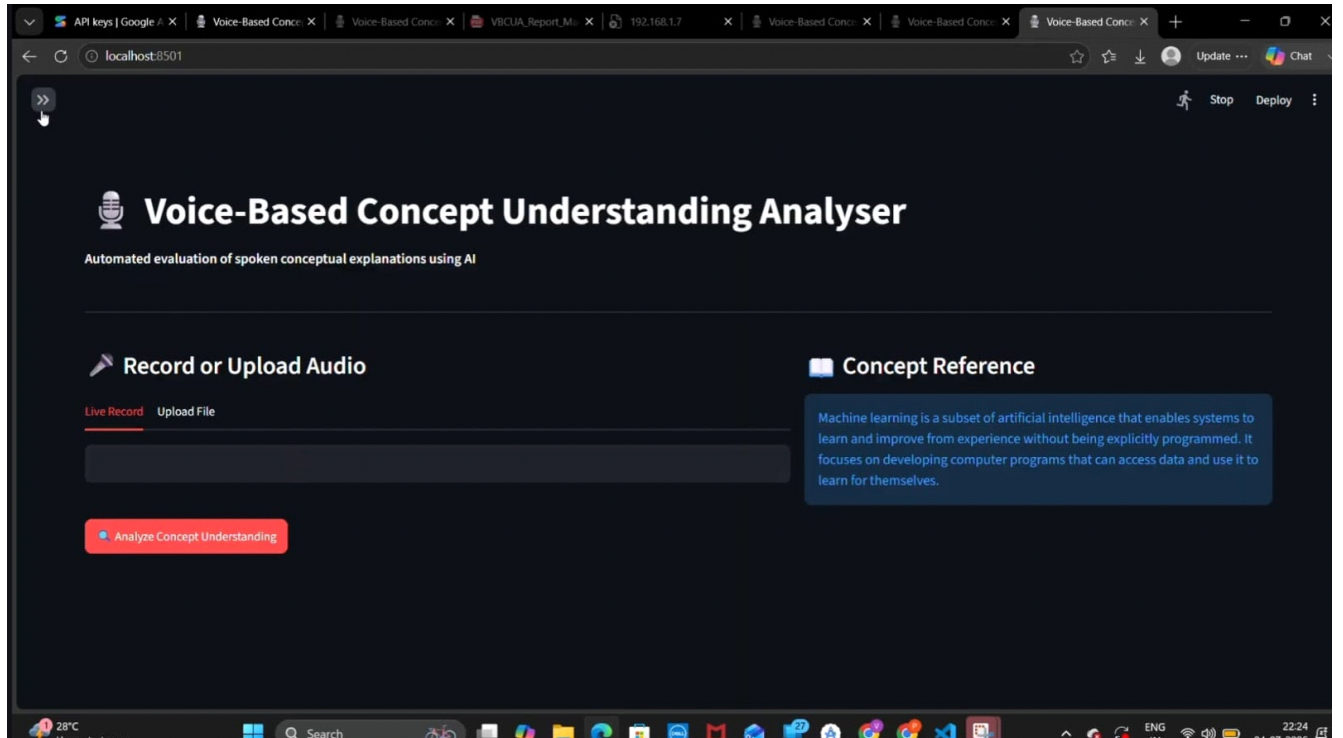
Description: Once `st.session_state.result` is populated, a horizontal rule and an "Analysis Results" subheader introduce a two-column row: the left column shows the full transcribed text under a "Transcribed Explanation" label; the right column shows a large, color-coded score heading (e.g., a green "85/100" for a Strong Understanding result) with the matching colored-circle emoji and level label directly beneath it. A three-column metrics row follows, using `st.metric` to display Semantic Similarity, Filler Words, and Fluency Score side by side. If waveform bytes were successfully generated, the dark-themed Matplotlib waveform PNG is displayed full-width via `st.image()` immediately below the metrics. A "Download Full PDF Report" button then serves the ReportLab-generated PDF, and a final "Clear Results" button lets the user dismiss the current result panel (without removing it from the sidebar's Attempt History).

User Guide: Voice-Based Concept Understanding Analyser

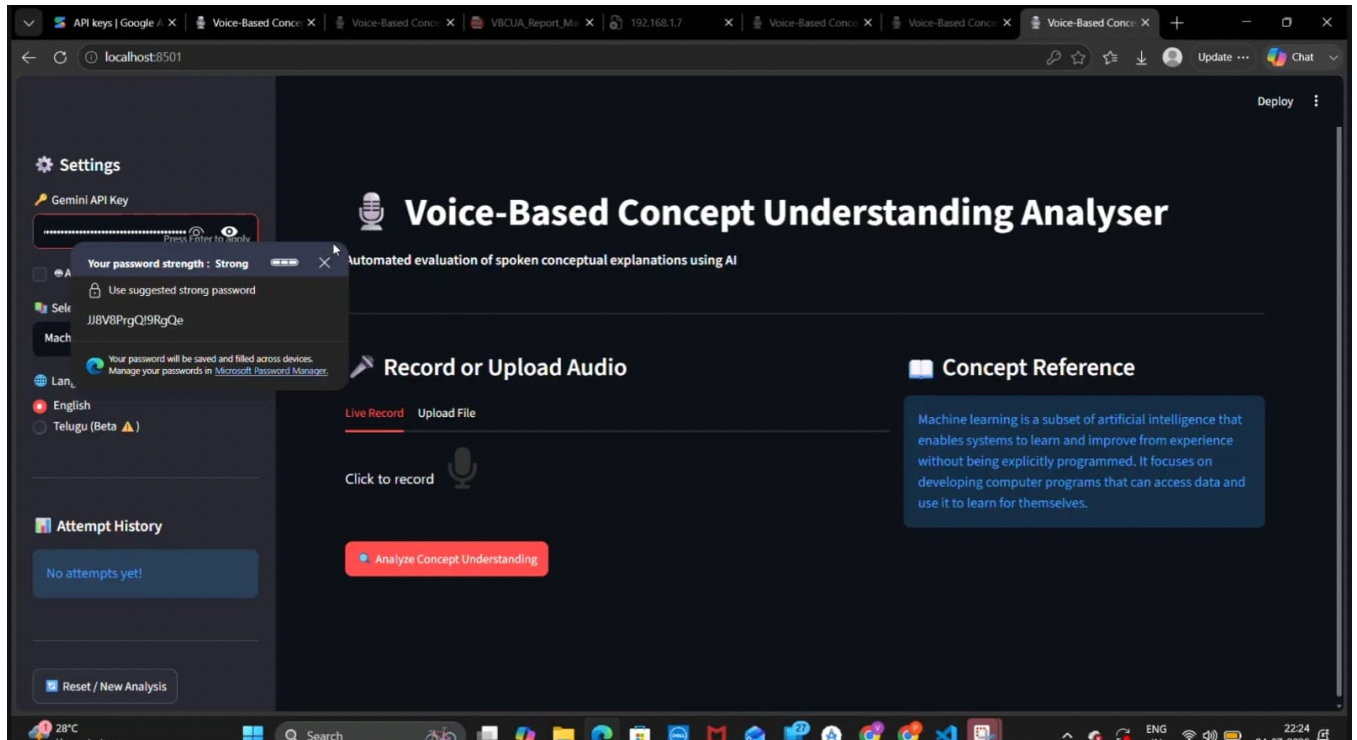
The step-by-step guide below walks a new user through a full session end-to-end, with a screenshot from an actual local run illustrating each step.

Step 1: Initial Setup and Configuration

- Open the application interface via your browser at localhost:8501.
- Click the expand arrows (>>) in the top-left corner to open the Settings sidebar.
- Enter your Gemini API Key into the designated password field.
- Select your target Language (e.g., English or Telugu (Beta)).



The app on first load, sidebar collapsed, Machine Learning reference shown on the right.



Sidebar expanded: Gemini API Key field, Language options, and an empty Attempt History.

Step 2: Selecting the Evaluation Topic

Before recording or uploading, choose how the AI should evaluate your subject material:

- **Manual Selection:** Click on the Select Concept Topic dropdown menu to choose from pre-configured domains, including Machine Learning, Cloud Computing, Deep Learning, Artificial Intelligence, Neural Networks, Data Science, or Natural Language Processing.
- **Automated Selection:** Alternatively, toggle the Auto Detect Topic (Gemini) checkbox to let the AI dynamically identify your topic from your speech context.



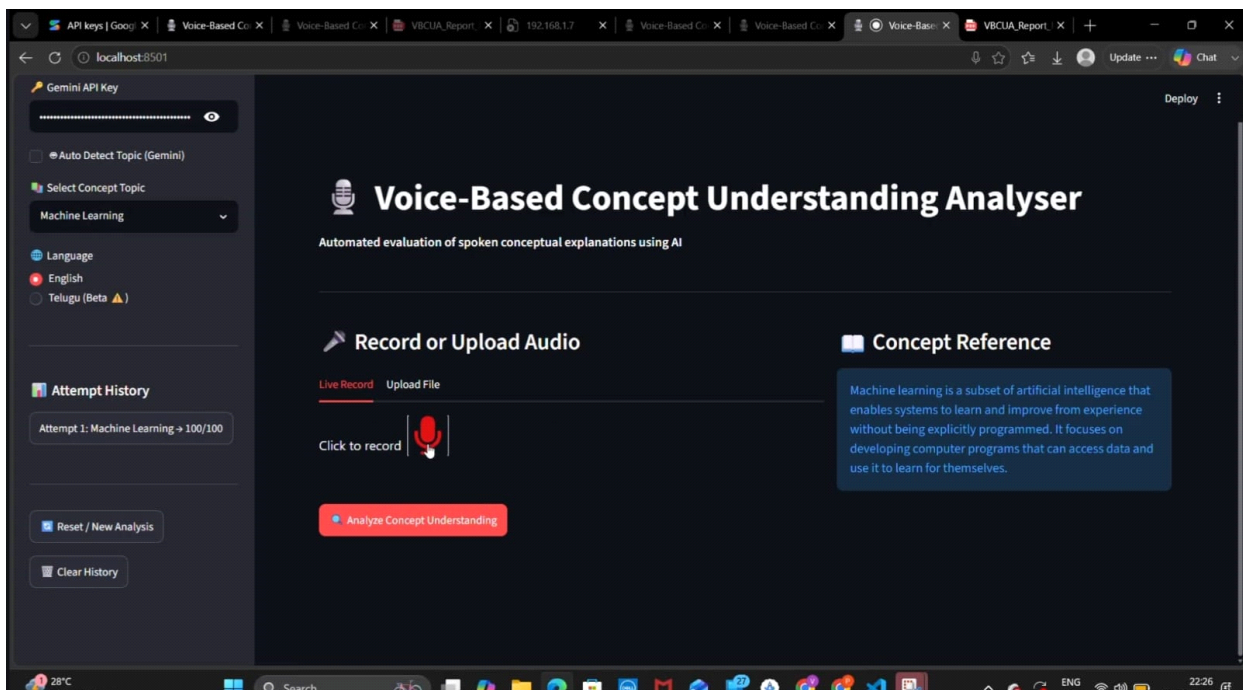
The Select Concept Topic dropdown expanded, showing all pre-configured domains.

Step 3: Providing Audio Input

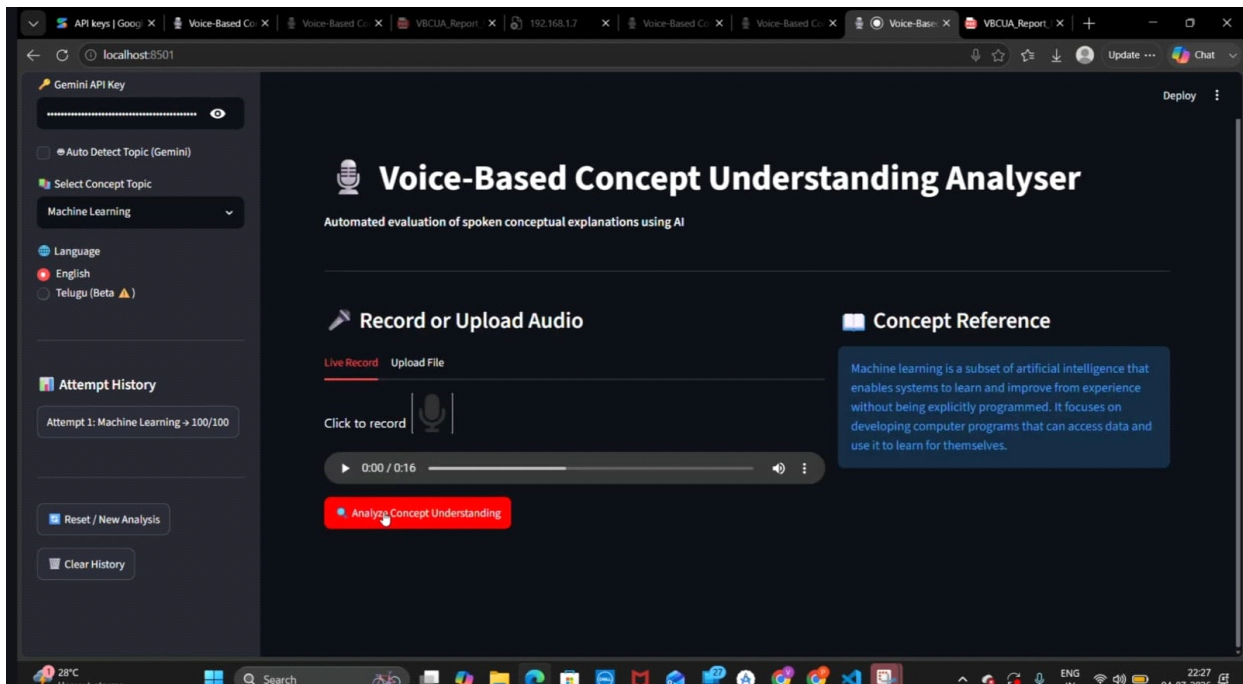
You can provide your spoken conceptual explanation via two separate methods:

Option A: Live Recording

- Select the Live Record tab.
- Click the microphone icon to begin speaking your explanation directly into your system's microphone.
- Click the icon again to stop recording. A built-in audio playback bar will appear, allowing you to play back your raw recording before submission.



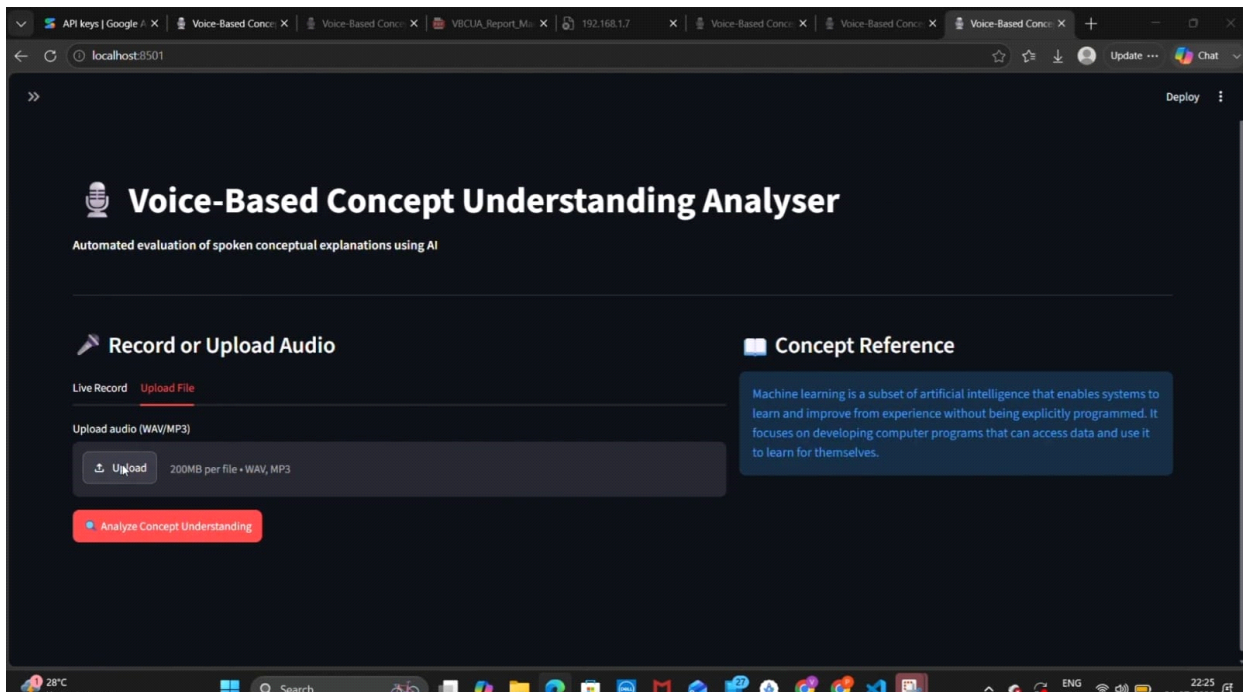
Live Record tab active, ready to capture audio from the microphone.



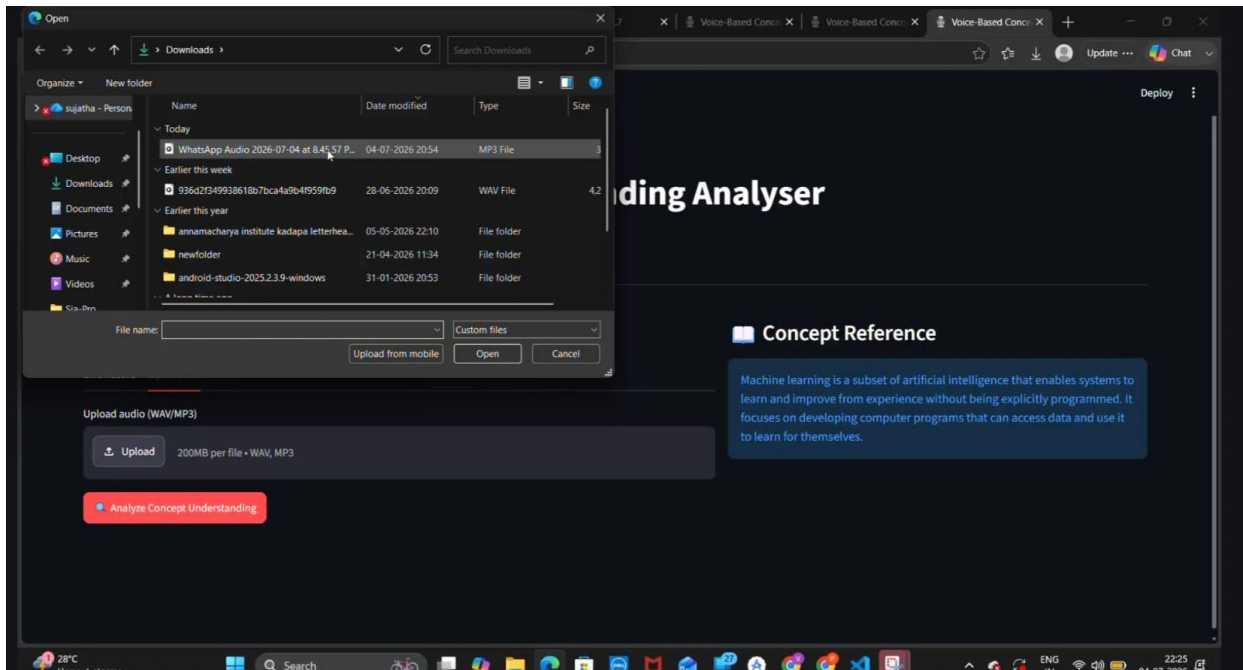
Recording captured — the inline audio playback bar (0:00 / 0:16) appears above the Analyze button.

Option B: File Upload

- Navigate to the Upload File tab under the Record or Upload Audio section.
- Click the Upload button to open your local file explorer dialog.
- Select an audio file (supporting standard formats like WAV or MP3 under 200MB) and confirm selection.



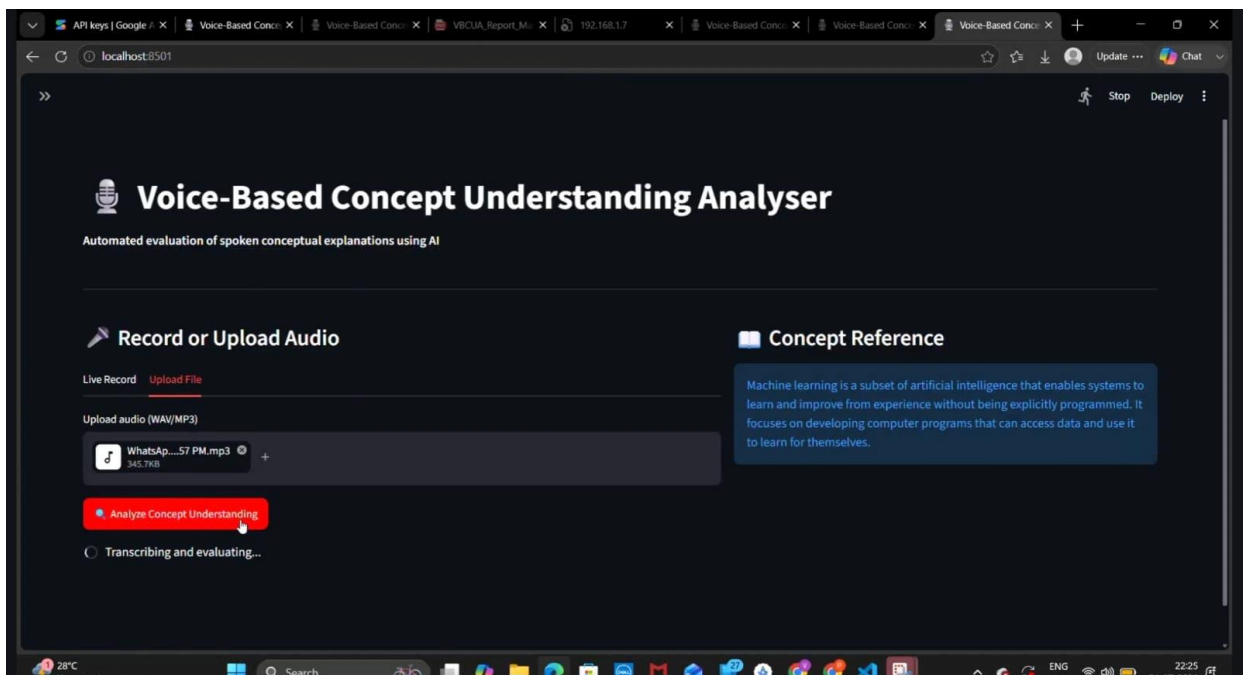
Upload File tab, showing the 200MB WAV/MP3 upload control.



The system file explorer dialog used to select an audio file to upload.

Step 4: Running the Analysis

- Once your audio source is finalized, click the red Analyze Concept Understanding button.
- The application will display a Transcribing and evaluating... status bar while processing.

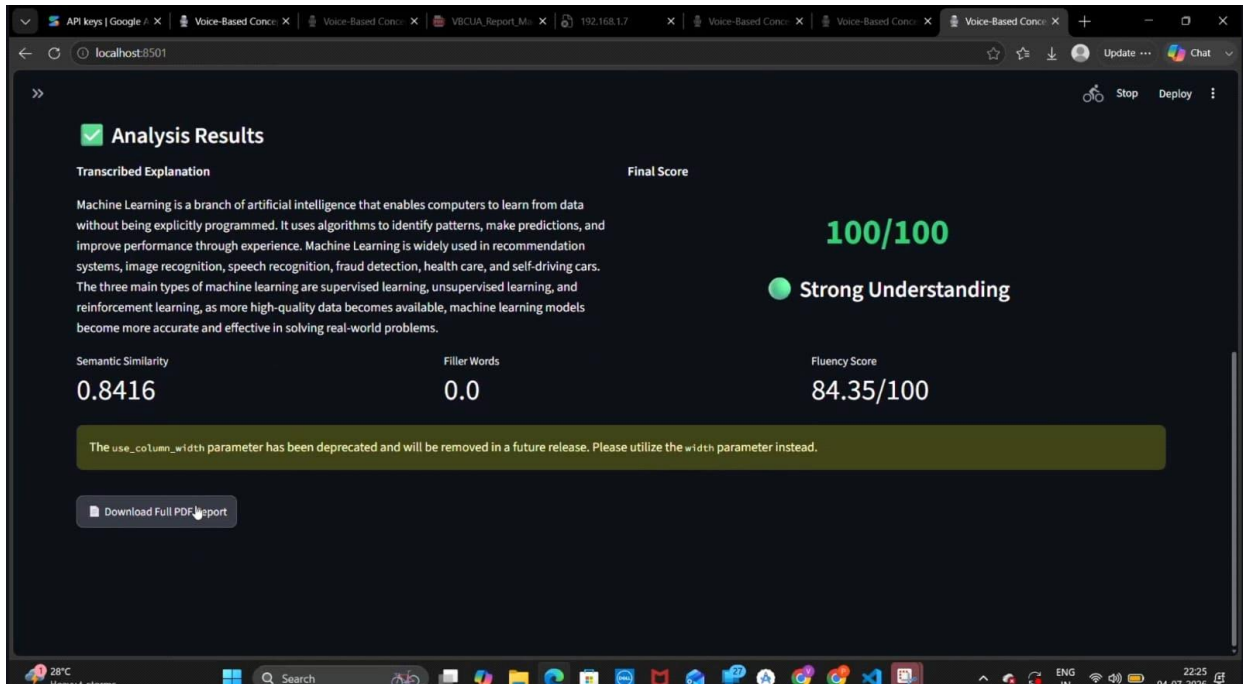


Uploaded file attached and Analyze clicked — the "Transcribing and evaluating..." status is shown.

Step 5: Reviewing Results and Metrics

- View the Analysis Results screen generated upon completion.
- Review the core metrics evaluated by the AI engine:
 - **Transcribed Explanation:** The full text parsed from your speech.

- **Final Score:** An overall performance rating (e.g., 100/100 - Strong Understanding).
- **Semantic Similarity:** Alignment accuracy between your response and the concept reference core baseline.
- **Filler Words:** Total count of spoken filler terms detected.
- **Fluency Score:** A performance rating measuring delivery pacing and clarity.



Analysis Results: Final Score 100/100 (Strong Understanding), with Semantic Similarity, Filler Words, and Fluency Score metrics.

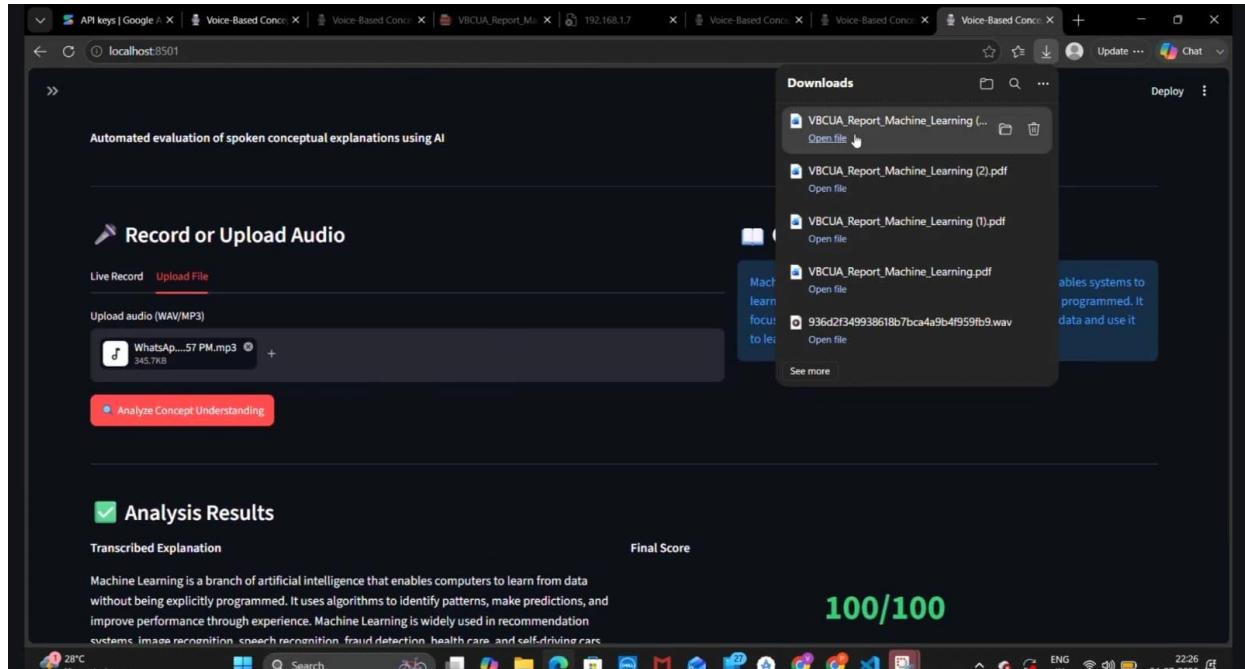
- Scroll down to review the corresponding Audio Waveform plot measuring signal amplitude over time.



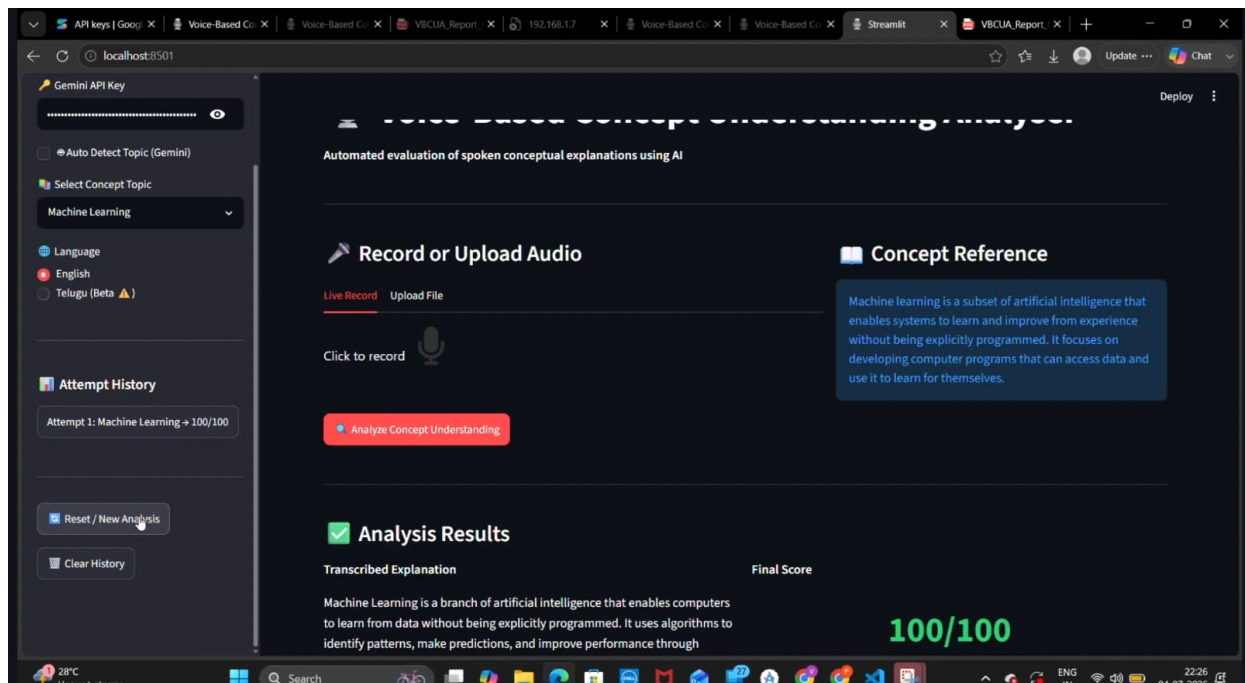
Audio Waveform plot rendered below the results, with Download Full PDF Report and Clear Results buttons.

Step 6: Exporting Reports and Tracking Progress

- Click the Download Full PDF Report button located below the results to export a structured document.
- Access the compiled evaluation sheet (e.g., VBCUA_Report_Machine_Learning.pdf) directly from your web browser's downloads manager.
- View your session history tracking log via the Attempt History panel on the lower section of the sidebar.
- To wipe parameters clean for a fresh entry, click Reset / New Analysis or wipe previous attempts using Clear History.



Browser downloads manager showing successive VBCUA_Report_Machine_Learning.pdf exports, each openable directly.



Sidebar Attempt History showing "Attempt 1: Machine Learning -> 100/100", with Reset / New Analysis and Clear History buttons.

7. Testing & Quality Assurance

The repository includes two plain, assert-based test scripts (not a pytest suite) under "6. Project Testing": `test_functional.py`, which checks correctness of the pure-Python helper functions, and `test_performance.py`, which benchmarks how fast those same functions run. Both scripts import directly from the sibling "5. Project Development Phase" folder via a relative `sys.path.append` and are run as standalone scripts rather than through a test runner.

7.1 Functional Tests (`test_functional.py`, as committed)

```
def test_scoring_engine():
    audio = {"pause_ratio": 0.2, "rms_energy": 0.05}
    score, level = evaluate_understanding(0.8, 0.02, audio)
    assert score >= 80 and "Strong" in level

def test_semantic_similarity():
    score = semantic_similarity("machine learning is a subset of AI", "ML learns from data")
    assert 0 <= score <= 1

# ...plus test_filler_word_ratio(), test_poor_understanding(), test_reference_exists(),
test_fluency_score()
```

Run with: `cd "6. Project Testing" then python test_functional.py`

7.2 Performance Tests (`test_performance.py`, as committed)

```
def test_semantic_similarity_speed():
    start = time.time()
    for _ in range(10):
        semantic_similarity("Machine learning is a subset of AI", "ML learns from data")
    assert (time.time() - start) / 10 < 2.0

def test_stability():
    scores = [evaluate_understanding(0.7, 0.03, audio)[0] for _ in range(10)]
    assert len(set(scores)) == 1 # same inputs must always give the same score

# ...plus test_scoring_speed(), test_filler_detection_speed(), test_fluency_calculation_speed()
```

Run with: `cd "6. Project Testing" then python test_performance.py`

test_stability(): confirms `evaluate_understanding()` has no hidden non-determinism — a valid, cheap regression guard for a deterministic scoring path.

7.3 Coverage Gaps in the Current Test Suite

Being direct about what these two scripts do not yet cover is more useful than presenting them as complete:

- **No tests for `speech_to_text()`** itself (the Whisper-calling function), `extract_audio_features()`, `save_waveform()`, `auto_detect_topic()`, `get_gemini_feedback()`, or `generate_pdf_report()` — all six either require audio fixtures, a Gemini API key, or file-generation assertions that the current scripts don't attempt.
- **No Streamlit UI-level tests** (e.g., via `streamlit.testing.v1.AppTest`) verifying widget wiring, session-state transitions, or the sidebar history topic-mismatch bug noted in Milestone 3.
- **No tests use a real pytest runner or fixtures;** both scripts rely on top-level `print()` statements and `assert` for pass/fail signaling and are invoked directly with `python`, which works for a quick manual

check but won't integrate with CI test reporting (JUnit XML, coverage percentages, etc.) without being ported to pytest.

Suggested near-term addition (not yet in the repository): a small pytest-based `test_speech_to_text.py` using a short pre-recorded fixture .wav file checked into the test folder, asserting only that `speech_to_text()` returns `success: True` and a non-empty text field — this would close the biggest coverage gap (the Whisper integration path) without requiring a live Gemini key.

8. Security Considerations

As a locally run, single-user Streamlit app, the Voice-Based Concept Understanding Analyser has a small default attack surface, but the current implementation has a handful of concrete, fixable issues worth documenting plainly.

8.1 Findings Summary

Finding	Location	Risk / Impact	Recommendation
Hardcoded developer-specific filesystem path	speech_to_text.py, get_ffmpeg_path()	Low security risk, but a real portability/maintainability issue: exposes a specific local username/path structure in source control	Replace with an FFMPEG_PATH environment variable or drop the fallback entirely in favor of a clear error
Bare except: swallows all Gemini errors	semantic_eval.py, auto_detect_topic()	An invalid/expired API key, network failure, or unexpected response silently produces a wrong topic ("Machine Learning") rather than a visible error, which can mislead the user about their own analysis	Catch specific exceptions and surface a st.warning() so the user knows auto-detection failed
Gemini API key handled only in memory, per session	app.py sidebar st.text_input(type="password")	Reasonably safe by default (not written to disk, not logged) since it is never persisted, but there is also no key validation before use, so a malformed key silently triggers the same bare-except fallback above	Add a lightweight key-format check and a clear inline error message on first failed call
No audio size/duration limits before processing	app.py Analyze handler, audio_utils.py	A very large or very long upload will simply take a long time to process locally; on a constrained machine this could stall the single-threaded Streamlit session for other tabs/interactions	Add a duration check via librosa.get_duration() (or file-size check) before calling Whisper, with a clear st.error() if exceeded
Recorded-audio temp files are never cleaned up	app.py Analyze handler (recorded_path branch)	Every live recording creates a NamedTemporaryFile that is deliberately kept (so sidebar history can reference it) but is never deleted even when history is cleared or the session ends, leading to temp-directory growth over a long-running session	Track and delete stale recorded_path files when history entries referencing them are cleared, or clean up on app shutdown
No input length cap on the topic dropdown/text paths	app.py / semantic_eval.py	Low risk since topic is selected from a fixed dropdown, not freely typed, but any future free-text topic input (see Future Scope) would need explicit length/content validation before being interpolated into a Gemini prompt	Apply the same prompt-injection-aware handling used for the transcript if/when free-text topic input is added

Transcript text is interpolated directly into Gemini prompts	semantic_eval.py, auto_detect_topic() and get_gemini_feedback()	A user could speak adversarial phrasing intended to manipulate the Gemini response (e.g., asking it to output an unrelated topic or ignore the feedback instructions); since Gemini's output here only affects topic selection and free-text coaching (not the numeric score, which is rule-based), the blast radius of a successful injection is limited to cosmetic/feedback content	If this pathway is extended to affect the numeric score in the future, add the same schema-validation backstop pattern used elsewhere in this document before trusting any LLM-influenced score
--	---	--	---

8.2 What Is Already Handled Reasonably Well

- **No public exposure by default:** streamlit run app.py binds to localhost; there is no Ngrok tunnel or public deployment step in this repository, so the default attack surface is limited to the local machine and local network.
- **The numeric score cannot be manipulated via prompt injection:** because evaluate_understanding() is a pure, deterministic function over similarity/filler/pause/energy values and never calls an LLM, there is no path for adversarial transcript content to directly inflate or deflate the final 0-100 score — only the optional topic-detection and free-text feedback are LLM-influenced.
- **The Gemini key is never written to disk or logged** in the current code; it lives only in Streamlit's session state for the duration of the browser session, which is a reasonable default for a local single-user tool.
- **File type is constrained** at the uploader (type=["wav", "mp3"]), reducing (though not eliminating) the chance of a wildly unexpected file reaching Whisper/Librosa.

8.3 Recommended Hardening Before Broader Distribution

- Remove the hardcoded personal ffmpeg path and replace it with environment-variable configuration, so the repository does not implicitly disclose a specific developer's machine layout.
- Replace bare except: blocks in semantic_eval.py with specific exception handling plus visible Streamlit warnings, so failures in the optional Gemini path are never silently mistaken for a real result.
- Add basic audio duration/size validation ahead of the Whisper and Librosa calls to bound worst-case local processing time.
- If the app is ever adapted from single-user-local to a shared/hosted deployment, revisit every finding above under a multi-user threat model — in particular, the current sidebar-entered Gemini key pattern and the unbounded temp-file accumulation are acceptable for one person on one machine but would need real per-user isolation and cleanup in a shared environment.

9. Conclusion

The Voice-Based Concept Understanding Analyser is a working, self-contained Streamlit application that turns a spoken explanation into a measurable understanding score without requiring any cloud infrastructure. Its core value proposition — record yourself explaining a concept, get an objective score back — is delivered end-to-end: local Whisper transcription keeps raw audio on-device, Sentence-BERT gives an objective similarity signal against a reference explanation, and a transparent, auditable rule-based formula (not an opaque LLM judgment) produces the final score. Gemini is layered on top as a genuinely optional enhancement, so the app is fully usable without any API key or ongoing cost.

This documentation deliberately reflects the codebase as it actually stands rather than an idealized version of it. That includes the real architecture (Section 3, with the proposed ER diagram for the not-yet-built persistence layer), the real UI walkthrough with real screenshots (Section 6), the real test coverage and its gaps (Section 7), and a plain-language account of current limitations and small bugs — the hardcoded ffmpeg path, the silent Gemini fallback, the history/result topic mismatch, and the lack of persistent storage (Section 8, Milestone 6). None of these are blockers to using the app locally today; they are the concrete, prioritized punch list for hardening it.

Taken together, the project demonstrates a coherent design decision worth calling out explicitly: keeping the **numeric score deterministic and rule-based** while using AI (Whisper, Sentence-BERT, and optionally Gemini) for the parts that genuinely need it — transcription, semantic comparison, and free-text coaching. That split is what keeps the tool fast, reproducible, cheap to run, and safe from prompt-injection affecting the actual grade, while still giving users AI-quality feedback where it matters most.